Code Mode for Governed Enterprise CRM Access: An Empirical Comparison of Sandboxed Program Execution and Per-Call Tool Use over Salesforce

Srinivasa Pusuluri
https://www.linkedin.com/in/pusulurisrinivasa/

November 2026

Abstract

Large language model (LLM) agents usually access enterprise systems through one tool call per operation, with every intermediate result passing through the model's context. *Code Mode* replaces this pattern: the model writes a single program against typed APIs, and the program runs in an isolated sandbox that calls the tools on the model's behalf. We present an open implementation of Code Mode for Salesforce, exposed to Claude through the Model Context Protocol (MCP), together with a host-side governance layer: per-user OAuth, role-based access control, field-level security, human approval for writes, redaction, and a hash-chained audit log. We compare Code Mode with a deliberately strong per-call baseline on five identical CRM workflows (30 runs each) and run the platform against a live Salesforce Developer Edition org. Measured results: Code Mode reduced model round trips by 33–50% and tool invocations by 67–94%, with identical Salesforce API consumption and 100% task success in both modes. Estimated results: input tokens fell by 7.5–35.1% when API type declarations were preloaded, but *rose* by 11–57% when the model also had to call a discovery tool, and output tokens rose by 44–86% on four of the five workflows because generating a program costs more than emitting a short tool call. Under a stated latency model, Code Mode was slower end to end on four of five small workflows and 18% faster only on the fan-out workflow. We also report two defects that only the evaluation exposed: lost audit events under parallel tool calls, and a synchronous infinite loop that stalled the shared gateway despite a configured CPU limit. We conclude that Code Mode's clearest benefits are fewer round trips and tool calls, and that token and latency savings depend on fan-out, payload size and how types are delivered.

1 Introduction

Agents that read and write CRM data are a common enterprise use of LLMs. The prevailing integration pattern exposes each operation (search accounts, list cases, create a case) as a separate tool. A task that needs ten lookups then needs several model turns, and each raw record returned by a tool is read by the model.

Code Mode, popularised by Cloudflare [1] and discussed by Anthropic as code execution with MCP [2], inverts this. The model is shown TypeScript declarations for the available operations and writes one program; the program runs in a sandbox and returns a compact result. The idea is related to executable code actions for agents [3], but brings a new constraint in enterprise settings: generated code is untrusted, while the data it touches is governed by access controls that must still hold.

This paper makes four contributions:

1. An open implementation of a Code Mode gateway for Salesforce, built on Cloudflare Dynamic Workers and MCP, with a single host-side checkpoint through which every sandbox call passes (Section 3).
2. A controlled comparison against a strong per-call baseline on five workflows, separating *measured* quantities (round trips, tool calls, API calls, gateway latency, success) from *estimated* ones (tokens, model time) (Sections 4–5).
3. A live-org check that the same programs run unchanged against a real Salesforce org under user-scoped OAuth.
4. Two failure findings and the mitigations we applied, which we believe generalise to other Code Mode deployments (Section 6).

2 Background

Tool use. ReAct [4] interleaves reasoning and actions; Toolformer [5] trains models to call APIs. In both, each action is a separate model decision.

Code as action. CodeAct [3] shows that letting agents emit executable Python can raise success rates and reduce the number of turns compared with JSON tool calls.

MCP. The Model Context Protocol [6] standardises how clients discover and call tools on remote servers. Code Mode can be offered over MCP by exposing a small number of meta-tools instead of one tool per operation.

3 System design

Interface. The gateway exposes two MCP tools. `discover_capabilities` returns, for the authenticated user only, TypeScript declarations of the 29 registered operations they may call (sales, service, CPQ, metadata, Data Cloud, Agentforce, and a second system, NetSuite). `execute_code` accepts one TypeScript program.

Execution. Programs are type-stripped, parsed, statically screened, and run in a fresh Cloudflare Dynamic Worker isolate with outbound network disabled and no credentials or environment bindings. Tests use a QuickJS WebAssembly isolate implementing the same executor interface.

Single checkpoint. Each `sf.*` call inside the sandbox is a remote procedure call to the host. One function performs, in order: role-based authorisation and allowlisting; per-user rate limits and per-execution call budgets; schema validation with Zod; a human-approval gate for writes; field-level-security filtering using the user's own object describe; query construction from validated identifiers and escaped literals (no raw query path exists); redaction of sensitive values; and an append to a hash-chained audit log.

Identity. Each Claude user authorises their own Salesforce account through OAuth 2.0 with PKCE. Tokens are encrypted at rest with AES-256-GCM. Salesforce therefore enforces that user's object permissions, field permissions and sharing rules; the gateway never acts as a shared integration user.

Writes. A write first returns `approval_required`. A different user approves the exact tool, arguments and requester (bound by a hash). Re-running the program consumes the approval once; any change to the arguments requires a new approval.

4 Method

Workflows. Five CRM tasks with different shapes: *Account 360* (search, then three parallel reads), *Sales pipeline* (accounts, then aggregate and list opportunities), *Customer risk* (accounts with custom fields, open cases, opportunity trends), *Service intelligence* (three dependent pages of cases), and *Cross-system analysis* (eight accounts, each joined with two NetSuite calls).

Data. A deterministic mock Salesforce org (8 accounts, 14 contacts, 16 opportunities, 14 cases) that simulates per-user CRUD, field-level security and sharing.

Approaches. (1) *Traditional MCP*: each operation is a separate tool. We give this baseline every advantage: all independent calls in a turn are issued in parallel, and object metadata is cached across calls. (2) *Code Mode, discover + execute*: the model calls `discover_capabilities`, then `execute_code`. (3) *Code Mode, execute only*: type declarations are preloaded in the model's context (for example via a project instructions file), so only `execute_code` is called. All three run against the same gateway, the same user and the same policies. Code Mode programs are the hand-written reference programs shipped with the system.

Measured. Model round trips (tool-calling turns plus one final answer turn), tool invocations at the MCP boundary, Salesforce API calls, gateway latency, and success (the program's result passes a workflow-specific check; for the baseline, every planned call succeeds). Each cell is run 30 times.

Estimated. No live model is called. Tokens are estimated as UTF-8 characters divided by 4 over the exact payloads that would be exchanged, applied identically to all approaches; the system prompt and final answer are fixed at 400 and 250 tokens. Model time per turn is modelled as $800\,\text{ms} + \text{output}/60\,\text{tok/s} + \text{input}/8000\,\text{tok/s}$. These values are assumptions, not measurements.

5 Results

5.1 Benchmark

Table 1 reports all cells. Salesforce API calls were identical across approaches for every workflow, which confirms that the approaches performed the same work. All 450 runs succeeded.

Round trips and tool calls (measured). Code Mode needed two model turns for every workflow, against three or four for the baseline (Table 2). Tool invocations fell to one per task; the largest reduction was the fan-out workflow, from 18 to 1.

Tokens (estimated). With preloaded types, input tokens fell by 7.5–35.1%; the saving grew with the number of dependent steps, since each baseline turn re-reads all earlier results. When discovery was called in-band, its response became the largest item in the context, and input tokens rose by 11–57% (Figure 1). Output tokens rose by 44–86% on four workflows because a program is longer than a tool call.

Table 1: Five workflows, 30 runs per cell. RT = model round trips, Calls = tool invocations, API = Salesforce API calls (all measured). In/Out = tokens, Time = completion time (estimated). Lat = median gateway latency (measured, ms).

Workflow	Approach	RT	Calls	API	In (est)	Out (est)	Lat	Time (est, s)
Account 360	Traditional	3	4	8	14,903	346	0.48	10.0
	CM discover	3	2	8	23,450	662	6.35	16.4
	CM execute	2	1	8	13,781	644	2.39	14.1
Sales pipeline	Traditional	3	3	5	16,841	396	0.56	11.1
	CM discover	3	2	5	23,516	587	4.84	15.1
	CM execute	2	1	5	13,847	569	2.15	12.8
Customer risk	Traditional	3	3	6	17,893	370	0.45	10.8
	CM discover	3	2	6	23,299	641	5.27	16.0
	CM execute	2	1	6	13,630	623	2.41	13.7
Service intel.	Traditional	4	3	4	20,871	310	0.28	11.0
	CM discover	3	2	4	23,223	553	4.17	14.5
	CM execute	2	1	4	13,554	535	1.90	12.2
Cross-system	Traditional	3	18	4	17,611	677	1.06	15.9
	CM discover	3	2	4	24,154	594	4.95	15.3
	CM execute	2	1	4	14,485	576	2.89	13.0

Table 2: Change of Code Mode (execute only) relative to the traditional baseline. Negative is a saving.

Workflow	Round trips	Tool calls	Input tok. (est)	Output tok. (est)	Time (est)
Account 360	−33%	−75%	−7.5%	+86%	+40%
Sales pipeline	−33%	−67%	−17.8%	+44%	+15%
Customer risk	−33%	−67%	−23.8%	+68%	+27%
Service intel.	−50%	−67%	−35.1%	+73%	+11%
Cross-system	−33%	−94%	−17.7%	−15%	−18%

Time (estimated). Under our latency model, the higher output cost outweighed the saved turns on four workflows (11–40% slower). Code Mode was 18% faster only on the cross-system workflow.

Gateway latency (measured). Code Mode added 1.9–2.9 ms of median gateway time per task in the local QuickJS runtime, which is small relative to model time.

5.2 Live Salesforce org

We then connected the gateway in live mode to a Salesforce Developer Edition org through per-user OAuth with PKCE and ran the same programs without modification. Table 3 lists single runs; latency includes network round trips to Salesforce.

In the same session we ran security probes against the live gateway: missing and malformed tokens were rejected; a viewer role could not execute code; a quote-breaking search string matched no records and an injected identifier was rejected by validation; an analyst could not write; an administrator's write paused for approval with no record created; network calls were rejected; sensitive values were redacted; and the audit chain verified across 160 events.

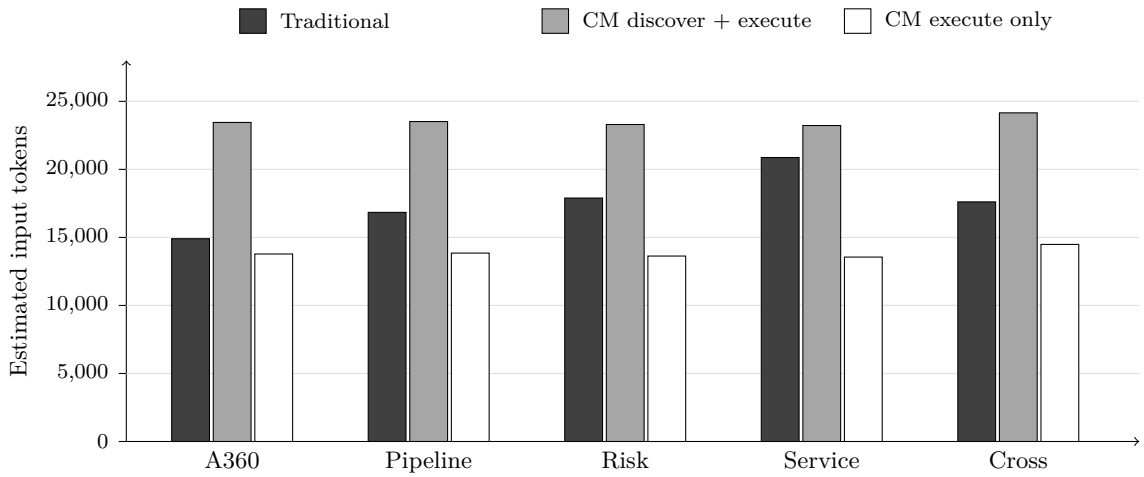


Figure 1: Estimated input tokens. Preloading type declarations makes Code Mode cheaper than the baseline; calling discovery every time makes it more expensive on these small workflows.

Table 3: Single runs against a live Developer Edition org (Dynamic Worker runtime under local `wrangler dev`).

Program	Tool calls	API calls	Latency (ms)	Outcome
Account 360	4	8	1,558	success
Four object lists in parallel	4	8	1,211	36/31/31/27 rows
Pipeline summary	1	2	619	13 open opportunities
Sales pipeline	3	5	1,495	success
Customer risk	3	6	1,065	ran; custom fields absent in org
Service intelligence	1	2	539	success
Opportunity intelligence	2	3	435	success
CPQ analysis	1	1	94	refused: Quote not enabled
Knowledge assistant	3	1	167	refused: Knowledge not enabled

5.3 Test suite

The repository contains 156 automated tests (unit, MCP protocol, integration and security), all passing. Security tests cover object, field and record access, query injection, credential leakage, network, file and process access, attempts to bypass static checks, infinite loops, memory exhaustion, oversized results, unauthorised writes, rate limits and audit integrity.

6 Findings from the evaluation

Lost audit events under parallel calls. The first benchmark run reported half the expected API calls. The cause was that parallel tool calls from one program interleaved read-modify-write sequences in the audit log, losing entries; the rate limiter, approval store and idempotency store had the same race. We serialised each with a keyed mutex and added a regression test. Code Mode makes parallel host calls the normal case, so any shared host state must be safe under concurrency.

A synchronous loop stalled the gateway. In the Dynamic Worker runtime under local development, a program containing a synchronous infinite loop blocked the whole gateway, including

its health endpoint. The executor's wall-clock timeout cannot fire while the isolate is busy, and a configured CPU limit did not stop it locally. Because all governance state lives in one Durable Object, one such program can stall every user. We added the CPU limit, a static check for obviously unbounded loops, and a deployment probe; we could not confirm enforcement on the production runtime. Deployments that accept untrusted code should run sandbox execution outside shared state.

Discovery payloads are expensive. Our first discovery format repeated each schema both as JSON Schema and as TypeScript, which roughly doubled its size. A compact default reduced it, but it remains the largest single payload. How types reach the model matters more to token cost than the execution model itself.

7 Threats to validity

No live model was used for the benchmark, so token counts and times are estimates, and answer quality was not measured. The reference programs were hand-written; a model may write longer or failing programs, which would reduce success rates and increase cost. The baseline was given parallel calls and shared caching, which favours it. The mock data is small; larger result sets would increase baseline input tokens, since raw records enter the context, while Code Mode returns only aggregates, but we did not measure this. Live-org results are single runs on one Developer Edition org with sparse data.

8 Conclusion

On identical workflows, Code Mode consistently reduced model round trips (33–50%) and tool invocations (67–94%) without changing the work done against Salesforce. Token and latency benefits were conditional: they appeared when types were preloaded and when tasks fanned out, and reversed for small tasks when discovery was called in-band. A governed implementation is practical, with user-scoped identity, a single checkpoint and human approval for writes, but the sandbox's resource limits and the concurrency of host state need explicit verification. Future work should repeat the comparison with live models, larger datasets and model-written programs.

Reproducibility

Code, workflows, raw benchmark output (`tests/evaluations/results/benchmark.json`) and tests are in the accompanying repository.

```
npm install && npm test # 156 tests
npm run bench -- 30 # regenerates Tables 1-2
npm run example -- account-360 # one workflow on the mock org
```

Software: `@cloudflare/codemode` 0.5.6, `@modelcontextprotocol/sdk` 1.32.2, Zod 4.6.7, Wrangler 4.160.0, Node.js 24.10.0.

Use of AI assistance

The software, benchmark harness and a draft of this manuscript were produced with the assistance of an AI coding agent (Claude). The authors reviewed the code, reran the experiments and take

responsibility for the content.

References

[1] K. Varda and S. Pai. Code Mode: the better way to use MCP. Cloudflare Blog, 2025. https://blog.cloudflare.com/code-mode/

[2] Anthropic. Code execution with MCP: building more efficient agents. Anthropic Engineering Blog, 2025. https://www.anthropic.com/engineering/code-execution-with-mcp

[3] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji. Executable code actions elicit better LLM agents. In *Proc. ICML*, 2024.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proc. ICLR*, 2023.

[5] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Proc. NeurIPS*, 2023.

[6] Anthropic. Model Context Protocol specification. https://modelcontextprotocol.io, 2024–2025.